INDIRA GANDHI NATIONAL OPEN UNIVERSITY

School of Computer and Information Sciences

Master of Science (Data Science and Analytics) — MSCDSA

JULY 2026 / SEMESTER I

MCSL-064

Data Structures using Python Lab

Lab assignment and practical write-up

Student particulars	Details
Name	
Enrolment number	
Programme	MSCDSA
Regional Centre	38 — Delhi 3 (Naraina)
Study Centre code / name	_____
Session	July 2026
Assignment number	MSCDSA(I)/L-064/Lab_Assign/26
Due date in the supplied paper	31 October 2026
Submission date	_____

Assessment: assignment questions 40 marks + lab record 40 marks + viva voce 20 marks = 100 marks. Assignment weightage: 30%.

Student signature: _____

The Regional Centre is recorded separately from the Study Centre. Fill the Study Centre and sign/date the report before submission.

Practical 1 | Matrix addition

Question 1 — Write a program in Python for addition of two matrices. (20 marks)

Aim: add two matrices element by element using nested loops, after checking that both matrices have the same order.

Principle

If A and B each have m rows and n columns, their sum C has the same order and C[i][j] = A[i][j] + B[i][j]. A 2 × 3 matrix can be added to another 2 × 3 matrix, but cannot be added to a 3 × 2 matrix.

Algorithm

1. Read the number of rows and columns and the entries of both matrices.
2. Reject empty matrices, uneven row lengths, nonnumeric entries, or different orders.
3. Visit each position (i, j), add A[i][j] and B[i][j], and store the sum in a new row.
4. Append each completed row to the result and display the three matrices.

A	B	A + B
4 −2 7	6 8 −1	10 6 6
0 5 3	9 −5 2	9 0 5

Manual check: the entry in row 1, column 2 is −2 + 8 = 6; the entry in row 2, column 2 is 5 + (−5) = 0. The inputs remain unchanged.

Time complexity is O(mn). The new sum matrix uses O(mn) space; only a constant amount of extra space is used besides the output. Python 3.12 was used; no external library is needed.

Practical 1 | code 1/2

File: matrix_addition.py. Full, selectable source; line continuations are indented.

```
"""MCSL-064, Q1: add two matrices of the same order."""

import argparse
from numbers import Real


def matrix_shape(matrix):
    if not matrix or not matrix[0]:
        raise ValueError("A matrix must have at least one row and one column.")
    columns = len(matrix[0])
    for row in matrix:
        if len(row) != columns:
            raise ValueError("All rows of a matrix must have the same length.")
        if any(not isinstance(value, Real) for value in row):
            raise TypeError("Matrix entries must be numbers.")
    return len(matrix), columns


def add_matrices(first, second):
    rows, columns = matrix_shape(first)
    if (rows, columns) != matrix_shape(second):
        raise ValueError("Addition requires matrices of the same order.")
    result = []
    for i in range(rows):
        row = []
        for j in range(columns):
            row.append(first[i][j] + second[i][j])
        result.append(row)
    return result


def show_matrix(name, matrix):
    print(name)
    for row in matrix:
        print("  " + " ".join(f"{value:7g}" for value in row))


def read_matrix(name, rows, columns):
    print(f"Enter {name}, one row at a time ({columns} numbers per row).")
    matrix = []
    for i in range(rows):
        values = [float(value) for value in input(f"Row {i + 1}: ").split()]
        if len(values) != columns:
            raise ValueError(f"Row {i + 1} must contain {columns} numbers.")
        matrix.append(values)
    return matrix


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--demo", action="store_true")
```

Practical 1 | code 2/2

File: matrix_addition.py. Full, selectable source; line continuations are indented.

```
    args = parser.parse_args()
    print("MCSL-064 | Q1 | MATRIX ADDITION")
    try:
        if args.demo:
            first = [[4, -2, 7], [0, 5, 3]]
            second = [[6, 8, -1], [9, -5, 2]]
        else:
            rows = int(input("Number of rows: "))
            columns = int(input("Number of columns: "))
            if rows <= 0 or columns <= 0:
                raise ValueError("Rows and columns must be positive.")
            first = read_matrix("A", rows, columns)
            second = read_matrix("B", rows, columns)
        result = add_matrices(first, second)
        show_matrix("Matrix A:", first)
        show_matrix("Matrix B:", second)
        show_matrix("A + B:", result)
    except (ValueError, TypeError) as error:
        print(f"Input error: {error}")
        return 1
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Practical 1 | Execution and checks

Python: matrix addition

```
python matrix_addition.py --demo
```

```
MCSL-064 | Q1 | MATRIX ADDITION
Matrix A:
        4      -2       7
        0       5       3
Matrix B:
        6       8      -1
        9      -5       2
A + B:
       10       6       6
        9       0       5
```

Actual execution transcript · Exit status: 0

Figure 1. Recorded stdout from the actual demonstration run; command shown in the capture.

Case	Input / condition	Expected outcome
Rectangular matrices	The two 2 × 3 matrices above	[[10, 6, 6], [9, 0, 5]]
Decimals	[[1.5]] + [[−0.25]]	[[1.25]]
Different orders	1 × 2 and 2 × 1	ValueError
Empty / uneven rows	[] or [[1, 2], [3]]	ValueError
Text entry	[["a"]] + [[1]]	TypeError

To supply your own values, run python matrix_addition.py without --demo. For example, choose 2 rows and 3 columns, then enter one space-separated row at each prompt. Invalid input is reported and the program returns a nonzero exit status.

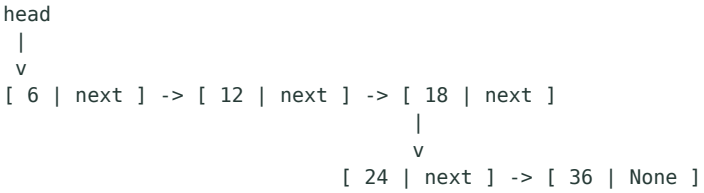
Result: the program produces the correct rectangular-matrix sum and handles incompatible input without modifying either matrix.

Practical 2 | Singly linked list

Question 2 — Write a program in Python for implementation of a singly linked list. (20 marks)

Aim: store values in nodes linked in one direction and implement insertion, deletion, search and traversal.

Each node stores a value and a next reference. The list has a head reference, and the last node has next = None. Node positions begin at zero. A separate size counter records how many nodes are present.



The diagram shows the list immediately after inserting 18 at position 2. The program does not use a built-in Python list to store the nodes; to_list() creates a temporary list only for displaying/checking their values.

Operation	Method	Time
Insert at front	Point the new node at the current head; replace head.	O(1)
Append	Traverse to the last node and connect the new node.	O(n)
Insert at position	Find the predecessor; connect predecessor → new node → successor.	O(n)
Delete value	Bypass the first matching node; replace head when necessary.	O(n)
Search	Traverse from head; return position or −1.	O(n)
Display	Traverse and format the node values.	O(n)

The stored list needs O(n) space. Insertion/deletion/search use O(1) working space. Displaying through to_list() and joining the text uses O(n) temporary space. Deleting an absent value returns False; invalid insertion positions raise IndexError.

Practical 2 | code 1/2

File: singly_linked_list.py. Full, selectable source; line continuations are indented.

```
"""MCSL-064, Q2: singly linked list with zero-based positions."""


class Node:
    def __init__(self, value):
        self.value = value
        self.next = None


class SinglyLinkedList:
    def __init__(self):
        self.head = None
        self.size = 0

    def prepend(self, value):
        node = Node(value)
        node.next = self.head
        self.head = node
        self.size += 1

    def append(self, value):
        if self.head is None:
            self.prepend(value)
            return
        current = self.head
        while current.next is not None:
            current = current.next
        current.next = Node(value)
        self.size += 1

    def insert(self, position, value):
        if position < 0 or position > self.size:
            raise IndexError("Position must be between 0 and the list size.")
        if position == 0:
            self.prepend(value)
            return
        previous = self.head
        for _ in range(position - 1):
            previous = previous.next
        node = Node(value)
        node.next = previous.next
        previous.next = node
        self.size += 1

    def delete(self, value):
        previous = None
        current = self.head
        while current is not None:
            if current.value == value:
                if previous is None:
                    self.head = current.next
```

Practical 2 | code 2/2

File: singly_linked_list.py. Full, selectable source; line continuations are indented.

```
52                  else:
53                      previous.next = current.next
54                  self.size -= 1
55                  return True
56              previous, current = current, current.next
57          return False
58
59      def search(self, value):
60          current = self.head
61          position = 0
62          while current is not None:
63              if current.value == value:
64                  return position
65              current = current.next
66              position += 1
67          return -1
68
69      def to_list(self):
70          values = []
71          current = self.head
72          while current is not None:
73              values.append(current.value)
74              current = current.next
75          return values
76
77      def __str__(self):
78          values = self.to_list()
79          return " -> ".join(map(str, values)) + " -> None" if values else "Empty list"
80
81
82  def main():
83      print("MCSL-064 | Q2 | SINGLY LINKED LIST")
84      linked = SinglyLinkedList()
85      print("Initially:", linked)
86      for value in (12, 24, 36):
87          linked.append(value)
88      print("Append 12, 24, 36:", linked)
89      linked.prepend(6)
90      print("Insert 6 at the front:", linked)
91      linked.insert(2, 18)
92      print("Insert 18 at position 2:", linked)
93      print("Search 18: position", linked.search(18))
94      print("Search 99:", linked.search(99), "(not found)")
95      for value in (24, 6, 36):
96          print(f"Delete {value}: {linked.delete(value)} | {linked}")
97      print("Delete missing value 99:", linked.delete(99))
98      print("Final size:", linked.size)
99
100
101 if __name__ == "__main__":
102     main()
```

Practical 2 | Execution and result

Python: singly linked list

```
python singly_linked_list.py
```

```
MCSL-064 | Q2 | SINGLY LINKED LIST
Initially: Empty list
Append 12, 24, 36: 12 -> 24 -> 36 -> None
Insert 6 at the front: 6 -> 12 -> 24 -> 36 -> None
Insert 18 at position 2: 6 -> 12 -> 18 -> 24 -> 36 -> None
Search 18: position 2
Search 99: -1 (not found)
Delete 24: True | 6 -> 12 -> 18 -> 36 -> None
Delete 6: True | 12 -> 18 -> 36 -> None
Delete 36: True | 12 -> 18 -> None
Delete missing value 99: False
Final size: 2
```

Actual execution transcript · Exit status: 0

Figure 2. Actual linked-list execution transcript. The demonstration covers front, middle and tail changes, present/absent search, and missing-value deletion.

Pointer changes

After insertion, the order is 6 → 12 → 18 → 24 → 36. Deleting 24 bypasses a middle node. Deleting 6 moves head to 12, and deleting 36 removes the tail. The final list is 12 → 18 → None and its size is 2.

Result: insertion, deletion, search and traversal work for a singly linked list. Searching for 18 returns position 2, whereas searching for 99 returns −1.

Verification and viva preparation

MCSL-064 functional tests

`python -m unittest -v`

```
test_duplicates_delete_first_occurrence (test_lab.LinkedListTests.test_duplicates_delete_first_occurrence
test_empty_and_single_node (test_lab.LinkedListTests.test_empty_and_single_node) ... ok
test_invalid_position_keeps_list_unchanged (test_lab.LinkedListTests.test_invalid_position_keeps_list_un
test_positions_and_deletion (test_lab.LinkedListTests.test_positions_and_deletion) ... ok
test_invalid_shapes (test_lab.MatrixTests.test_invalid_shapes) ... ok
test_non_numeric_entry (test_lab.MatrixTests.test_non_numeric_entry) ... ok
test_rectangular_addition_and_unchanged_inputs (test_lab.MatrixTests.test_rectangular_addition_and_uncha
test_single_fractional_entry (test_lab.MatrixTests.test_single_fractional_entry) ... ok

----------------------------------------------------------------------
Ran 8 tests in 0.000s

OK
```

Actual execution transcript · Exit status: 0

Figure 3. Eight functional tests executed successfully. Tests include empty and single-node lists, boundary positions, duplicates, incompatible matrices and numeric checks.

Viva question	Answer
Why must matrix orders match?	Each entry of A needs a corresponding entry of B at the same row and column.
Why use a new result matrix?	It preserves the two input matrices and stores the calculated sum separately.
What does head store?	A reference to the first node, or None when the list is empty.
Why save the successor when inserting?	Otherwise the rest of the chain can become unreachable.
Why is append O(n) here?	The implementation keeps head but no tail reference, so it traverses to the last node.
What happens with duplicate values?	Search finds the first match; delete removes only the first match.

Sources: the supplied MSCDSA-MCSL-064-2026.pdf question paper; Python 3 documentation (docs.python.org/3/tutorial/). The other student's solution was used to check topic coverage, not copied as a template. Programme-guide formatting still needs checking against an accessible official guide.